

Data Flow — DICOM 3D Brain Viewer

Three flows: files in, series list out, volume on screen.

1. Ingest — DICOM file → indexed volume

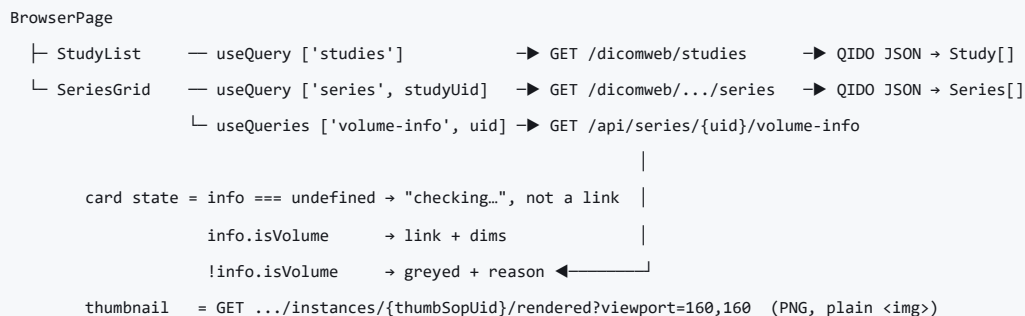
Runs on startup for `SAMPLES_DIR`, and per request for `POST /api/upload`. Same pipeline both ways (`ingest/indexer.py::ingest_files`).



Failure is per file, never per batch. A non-DICOM file, an undecodable transfer syntax or a malformed UID produces a `SkippedFile` with a reason; the rest of the upload still lands. `POST /api/upload` answers `200` with `{accepted, skipped[], studyUids[]}` even when some files were skipped.

Volume validation, first failure wins: sort method must be `geometry` → ≥ 3 slices → identical dimensions/pixel spacing → single orientation → uniform slice gaps within 1 % of the median. Each failure stores a human-readable reason.

2. Browse — studies and series



`['volume-info', seriesUid]` is the same query key the viewer uses, so clicking a card opens against already-cached info — no second request.

3. View — series → four synchronised panels

```

ViewerPage — useVolume(studyUid, seriesUid)
|
| status: info → metadata → loading → ready          (or blocked / error)
|
|
|
|      └─ loadVolume(volumeId, imageIds, onProgress, signal)
|
|      | volumeLoader.createAndCacheVolume → volume.load()
|      | IMAGE_VOLUME_MODIFIED           → progress
|      | IMAGE_VOLUME_LOADING_COMPLETED → resolve
|      | IMAGE_LOAD_ERROR                → release + reject "slice N failed"
|      | AbortSignal                     → release + reject AbortError
|
|      └─ GET ../metadata → instances (sorted) + X-Sort-Method
|
|          buildImageIds → wadors:<frameUrl> one per frame
|          seedMetadata  → wadors.metaDataManager (loader needs no extra requests)
|
|      └─ GET /api/series/{uid}/volume-info
|
|          isVolume === false → status 'blocked', reason shown, nothing loaded
|
|          estimatedBytes > 1 GB → window.confirm before loading
|
▼
ViewerLayout (status === 'ready')
createViewerLayout → 3 × ORTHOGRAPHIC (AXIAL/SAGITTAL/CORONAL) + 1 × VOLUME_3D
showVolume        → setVolumesForViewports(one volumeId, all four) + default 3D preset + resetCamera
createToolGroups  → MPR: Crosshairs/WindowLevel · Pan · Zoom · StackScroll
                  3D : TrackballRotate · Pan · Zoom
|
| Cornerstone fetches frames lazily: GET ../frames/{n} → multipart/related octet-stream
|
|
| └─ useViewportState per panel: VOI_MODIFIED · CAMERA_MODIFIED · VOLUME_NEW_IMAGE · IMAGE_RENDERED
|
|      → { sliceIndex, numSlices, voi, zoom } → ViewportOverlay

```

Crosshair sync is Cornerstone's `CrosshairsTool` bound to the three MPR viewport ids: dragging in one plane moves the camera of the other two, which emits `CAMERA_MODIFIED`, which updates their slice counters.

Teardown on unmount (series switch or back): abort the in-flight load, `destroyToolGroups()`, `destroyViewerLayout(engine)`, `releaseVolume(volumeId)`. `useVolume` 's started-latch is keyed to `seriesUid`, so a route param change loads the new series instead of reporting the old one as ready.

Error paths

Where	Behaviour
Backend unreachable	<code>ErrorBanner</code> + retry on the browser
Series not a volume	Viewer refuses to open; reason shown
One frame fails	Load aborted, partial volume released, slice named
Unknown UID	404 with {detail}
Upload too large / bad zip / path traversal	413 / 400, message shown in the dropzone
WebGL2 missing	Full-page "WebGL2 required" instead of a blank canvas